

Como ejemplo de un problema que requiere barreras, considere un problema común de relajación en física o ingeniería. Por lo general hay una matriz que contiene ciertos valores iniciales. Los valores podrían representar temperaturas en diversos puntos de una hoja de metal. La idea podría ser calcular cuánto tarda en propagarse el efecto de una flama colocada en una esquina a lo largo de la hoja.

Empezando con los valores actuales, se aplica una transformación a la matriz para obtener la segunda versión de ésta; por ejemplo, mediante la aplicación de las leyes de termodinámica para ver cuál es el valor de todas las temperaturas en un tiempo posterior, representado por ΔT . Después el proceso se repite una y otra vez, proporcionando las temperaturas en los puntos de muestreo como una función del tiempo, a medida que se calienta la hoja. El algoritmo produce una serie de matrices a través del tiempo.

Ahora imagine que la matriz es muy grande (por ejemplo, de 1 millón por 1 millón), por lo cual se requieren procesos en paralelo (posiblemente en un multiprocesador) para agilizar los cálculos. Distintos procesos trabajarán en distintas partes de la matriz, calculando los nuevos elementos de ésta a partir de los anteriores, de acuerdo con las leyes de la física. Sin embargo, ningún proceso puede empezar en la iteración $n + 1$ sino hasta que se complete la iteración n , es decir, hasta que todos los procesos hayan terminado su trabajo actual. La forma de lograr este objetivo es programar cada proceso para que ejecute una operación *barrier* después de que haya terminado su parte de la iteración actual. Cuando todos ellos hayan terminado, la nueva matriz (la entrada para la siguiente iteración) estará completa y todos los procesos se liberarán en forma simultánea para empezar la siguiente iteración.

2.4 PLANIFICACIÓN

Cuando una computadora se multiprograma, con frecuencia tiene varios procesos o hilos que compiten por la CPU al mismo tiempo. Esta situación ocurre cada vez que dos o más de estos procesos se encuentran al mismo tiempo en el estado listo. Si sólo hay una CPU disponible, hay que decidir cuál proceso se va a ejecutar a continuación. La parte del sistema operativo que realiza esa decisión se conoce como **planificador de procesos** y el algoritmo que utiliza se conoce como **algoritmo de planificación**. Estos terminos conforman el tema a tratar en las siguientes secciones.

Muchas de las mismas cuestiones que se aplican a la planificación de procesos también se aplican a la planificación de hilos, aunque algunas son distintas. Cuando el kernel administra hilos, por lo general la planificación se lleva a cabo por hilo, e importa muy poco (o nada) a cuál proceso pertenece ese hilo. Al principio nos enfocaremos en las cuestiones de planificación que se aplican a los procesos e hilos. Más adelante analizaremos de manera explícita la planificación de hilos y algunas de las cuestiones únicas que surgen. En el capítulo 8 trabajaremos con los chips multinúcleo.

2.4.1 Introducción a la planificación

En los días de los sistemas de procesamiento por lotes, con entrada en forma de imágenes de tarjetas en una cinta magnética, el algoritmo de planificación era simple: bastaba con ejecutar el siguiente trabajo en la cinta. Con los sistemas de multiprogramación, el algoritmo de planificación se

volvió más complejo debido a que comúnmente había varios usuarios esperando ser atendidos. Algunas mainframe todavía combinan los servicios de procesamiento por lotes y de tiempo compartido, requiriendo que el planificador decida si le toca el turno a un trabajo de procesamiento por lotes o a un usuario interactivo en una terminal. (Como punto y aparte, un trabajo de procesamiento por lotes puede ser una petición para ejecutar varios programas en sucesión, pero para esta sección sólo supondremos que es una petición para ejecutar un solo programa). Debido a que el tiempo de la CPU es un recurso escaso en estas máquinas, un buen planificador puede hacer una gran diferencia en el rendimiento percibido y la satisfacción del usuario. En consecuencia, se ha puesto gran esfuerzo en idear algoritmos de planificación astutos y eficientes.

Con la llegada de las computadoras personales, la situación cambió de dos maneras. En primer lugar, la mayor parte del tiempo sólo hay un proceso activo. Es muy poco probable que un usuario que entra a un documento en un procesador de palabras compile al mismo tiempo un programa en segundo plano. Cuando el usuario escribe un comando para el procesador de palabras, el planificador no tiene que esforzarse mucho en averiguar qué proceso ejecutar; el procesador de palabras es el único candidato.

En segundo lugar, las computadoras se han vuelto tan veloces con el paso de los años que la CPU es raras veces un recurso escaso. La mayoría de los programas para computadoras personales están limitados en cuanto a la velocidad a la que el usuario puede presentar los datos de entrada (al escribir o hacer clic), no por la velocidad con que la CPU puede procesarlos. Incluso las compilaciones, que en el pasado consumían muchos ciclos de la CPU, requieren unos cuantos segundos en la mayoría de los casos hoy en día. Aún y cuando se estén ejecutando dos programas al mismo tiempo, como un procesador de palabras y una hoja de cálculo, raras veces es importante saber quién se ejecuta primero, ya que el usuario probablemente está esperando a que ambos terminen. Como consecuencia, la planificación no es de mucha importancia en las PCs simples. Desde luego que hay aplicaciones que prácticamente se comen viva a la CPU; por ejemplo, para visualizar una hora de video en alta resolución, a la vez que se ajustan los colores en cada uno de los 108,000 cuadros (en NTSC) o 90,000 cuadros (en PAL) se requiere de un poder de cómputo a nivel industrial. Sin embargo, las aplicaciones similares son la excepción, en vez de la regla.

Al tratarse de servidores en red, la situación cambia en forma considerable. Aquí varios procesos compiten a menudo por la CPU, por lo que la planificación retoma importancia. Por ejemplo, cuando la CPU tiene que elegir entre ejecutar un proceso que recopila las estadísticas diarias y uno que atiende las peticiones de los usuarios, habrá usuarios más contentos si el segundo tipo de procesos tiene prioridad sobre la CPU.

Además de elegir el proceso correcto que se va a ejecutar a continuación, el planificador también tiene que preocuparse por hacer un uso eficiente de la CPU, debido a que la conmutación de procesos es cara. Para empezar, debe haber un cambio del modo de usuario al modo kernel. Después se debe guardar el estado del proceso actual, incluyendo el almacenamiento de sus registros en la tabla de procesos para que puedan volver a cargarse más adelante. En muchos sistemas, el mapa de memoria (por ejemplo, los bits de referencia de memoria en la tabla de páginas) se debe guardar también. Luego hay que seleccionar un nuevo proceso mediante la ejecución del algoritmo de planificación. Después de eso, se debe volver a cargar en la MMU el mapa de memoria del nuevo proceso. Por último, se debe iniciar el nuevo proceso. Además de todo esto, generalmente la conmutación de procesos hace inválida toda la memoria caché, por lo que tiene que volver a cargarse en forma diná-

mica desde la memoria principal dos veces (al momento de entrar al kernel y al salir de éste). Con todo, si se realizan muchas conmutaciones de procesos por segundo, se puede llegar a consumir una cantidad considerable de tiempo de la CPU, por lo cual se aconseja tener precaución.

Comportamiento de un proceso

Casi todos los procesos alternan ráfagas de cálculos con peticiones de E/S (de disco), como se muestra en la figura 2.38. Por lo general la CPU opera durante cierto tiempo sin detenerse, después se realiza una llamada al sistema para leer datos de un archivo o escribirlos en el mismo. Cuando se completa la llamada al sistema, la CPU realiza cálculos de nuevo hasta que necesita más datos o tiene que escribir más datos y así sucesivamente. Hay que tener en cuenta que algunas actividades de E/S cuentan como cálculos. Por ejemplo, cuando la CPU copia bits a una RAM de video para actualizar la pantalla, está calculando y no realizando operaciones de E/S, ya que la CPU está en uso. En este sentido, la E/S es cuando un proceso entra al estado bloqueado en espera de que un dispositivo externo complete su trabajo.

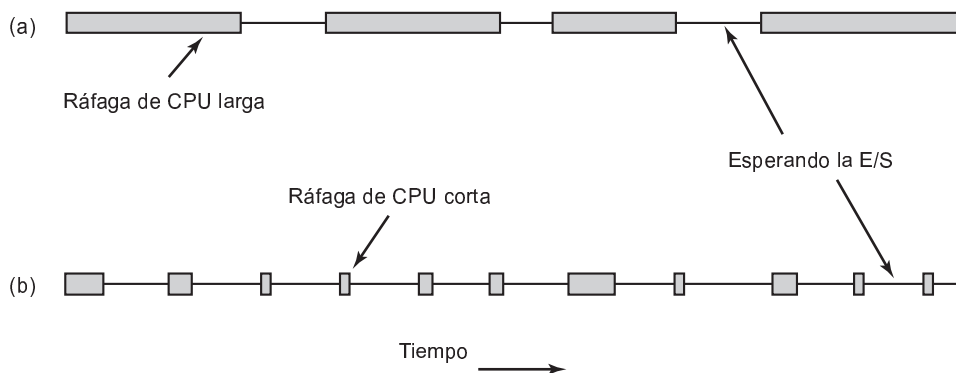


Figura 2-38. Las ráfagas de uso de la CPU se alternan con los periodos de espera por la E/S. (a) Un proceso ligado a la CPU. (b) Un proceso ligado a la E/S.

Lo importante a observar acerca de la figura 2-38 es que algunos procesos, como el que se muestra en la figura 2-38(a), invierten la mayor parte de su tiempo realizando cálculos, mientras que otros, como el que se muestra en la figura 2-38(b), invierten la mayor parte de su tiempo esperando la E/S. A los primeros se les conoce como **limitados a cálculos**; a los segundos como **limitados a E/S (I/O-bound)**. Por lo general, los procesos limitados a cálculos tienen ráfagas de CPU largas y en consecuencia, esperas infrecuentes por la E/S, mientras que los procesos limitados a E/S tienen ráfagas de CPU cortas y por ende, esperas frecuentes por la E/S. Observe que el factor clave es la longitud de la ráfaga de CPU, no de la ráfaga de E/S. Los procesos limitados a E/S están limitados a la E/S debido a que no realizan muchos cálculos entre una petición de E/S y otra, no debido a que tengan peticiones de E/S en especial largas. Se requiere el mismo tiempo para emitir la petición de hardware para leer un bloque de disco, sin importar qué tanto (o qué tan poco) tiempo se requiera para procesar los datos, una vez que lleguen.

Vale la pena observar que, a medida que las CPUs se vuelven más rápidas, los procesos tienden a ser más limitados a E/S. Este efecto ocurre debido a que las CPUs están mejorando con mucha mayor rapidez que los discos. Como consecuencia, es probable que la planificación de procesos limitados a E/S se convierta en un tema más importante en un futuro. La idea básica aquí es que, si un proceso limitado a E/S desea ejecutarse, debe obtener rápidamente la oportunidad de hacerlo para que pueda emitir su petición de disco y mantener el disco ocupado. Como vimos en la figura 2-6, cuando los procesos están limitados a E/S, se requieren muchos de ellos para que la CPU pueda estar completamente ocupada.

Cuándo planificar procesos

Una cuestión clave relacionada con la planificación es saber cuándo tomar decisiones de planificación. Resulta ser que hay una variedad de situaciones en las que se necesita la planificación. En primer lugar, cuando se crea un nuevo proceso se debe tomar una decisión en cuanto a si se debe ejecutar el proceso padre o el proceso hijo. Como ambos procesos se encuentran en el estado listo, es una decisión normal de programación y puede ejecutar cualquiera; es decir, el programador de procesos puede elegir ejecutar de manera legítima, ya sea el padre o el hijo.

En segundo lugar, se debe tomar una decisión de planificación cuando un proceso termina. Ese proceso ya no se puede ejecutar (debido a que ya no existe), por lo que se debe elegir algún otro proceso del conjunto de procesos listos. Si no hay un proceso listo, por lo general se ejecuta un proceso inactivo suministrado por el sistema.

En tercer lugar, cuando un proceso se bloquea por esperar una operación de E/S, un semáforo o por alguna otra razón, hay que elegir otro proceso para ejecutarlo. Algunas veces la razón del bloqueo puede jugar un papel en la elección. Por ejemplo, si *A* es un proceso importante y está esperando a que *B* salga de su región crítica, si dejamos que *B* se ejecute a continuación podrá salir de su región crítica y por ende, dejar que *A* continúe. Sin embargo, el problema es que el planificador comúnmente no tiene la información necesaria para tomar en cuenta esta dependencia.

En cuarto lugar, cuando ocurre una interrupción de E/S tal vez haya que tomar una decisión de planificación. Si la interrupción proviene de un dispositivo de E/S que ha terminado su trabajo, tal vez ahora un proceso que haya estado bloqueado en espera de esa operación de E/S esté listo para ejecutarse. Es responsabilidad del planificador decidir si debe ejecutar el proceso que acaba de entrar al estado listo, el proceso que se estaba ejecutando al momento de la interrupción, o algún otro.

Si un reloj de hardware proporciona interrupciones periódicas a 50 o 60 Hz o cualquier otra frecuencia, se puede tomar una decisión de planificación en cada interrupción de reloj o en cada *k*-ésima interrupción de reloj. Los algoritmos de planificación se pueden dividir en dos categorías con respecto a la forma en que manejan las interrupciones del reloj. Un algoritmo de programación **no apropiativo** (*nonpreemptive*) selecciona un proceso para ejecutarlo y después sólo deja que se ejecute hasta que el mismo se bloquea (ya sea en espera de una operación de E/S o de algún otro proceso) o hasta que libera la CPU en forma voluntaria. Incluso aunque se ejecute durante horas, no se suspenderá de manera forzosa. En efecto, no se toman decisiones de planificación durante las interrupciones de reloj. Una vez que se haya completado el procesamiento de la interrupción de reloj, se reanuda la ejecución del proceso que estaba antes de la interrupción, a menos que un proceso de mayor prioridad esté esperando por un tiempo libre que se acabe de cumplir.

Por el contrario, un algoritmo de planificación **apropiativa** selecciona un proceso y deja que se ejecute por un máximo de tiempo fijo. Si sigue en ejecución al final del intervalo de tiempo, se suspende y el planificador selecciona otro proceso para ejecutarlo (si hay uno disponible). Para llevar a cabo la planificación apropiativa, es necesario que ocurra una interrupción de reloj al final del intervalo de tiempo para que la CPU regrese el control al planificador. Si no hay un reloj disponible, la planificación no apropiativa es la única opción.

Categorías de los algoritmos de planificación

No es sorprendente que distintos entornos requieran algoritmos de planificación diferentes. Esta situación se presenta debido a que las diferentes áreas de aplicación (y los distintos tipos de sistemas operativos) tienen diferentes objetivos. En otras palabras, lo que el planificador debe optimizar no es lo mismo en todos los sistemas. Tres de los entornos que vale la pena mencionar son:

1. Procesamiento por lotes.
2. Interactivo.
3. De tiempo real.

Los sistemas de procesamiento por lotes siguen utilizándose ampliamente en el mundo de los negocios para realizar nóminas, inventarios, cuentas por cobrar, cuentas por pagar, cálculos de interés (en los bancos), procesamiento de reclamaciones (en las compañías de seguros) y otras tareas periódicas. En los sistemas de procesamiento por lotes no hay usuarios que esperen impacientemente en sus terminales para obtener una respuesta rápida a una petición corta. En consecuencia, son aceptables los algoritmos no apropiativos (o apropiativos con largos periodos para cada proceso). Este método reduce la conmutación de procesos y por ende, mejora el rendimiento. En realidad, los algoritmos de procesamiento por lotes son bastante generales y a menudo se pueden aplicar a otras situaciones también, lo cual hace que valga la pena estudiarlos, incluso para las personas que no están involucradas en la computación con mainframes corporativas.

En un entorno con usuarios interactivos, la apropiación es esencial para evitar que un proceso acapare la CPU y niegue el servicio a los demás. Aun si no hubiera un proceso que se ejecutara indefinidamente de manera intencional, podría haber un proceso que deshabilitara a los demás de manera indefinida, debido a un error en el programa. La apropiación es necesaria para evitar este comportamiento. Los servidores también entran en esta categoría, ya que por lo general dan servicio a varios usuarios (remotos), todos los cuales siempre tienen mucha prisa.

En los sistemas con restricciones de tiempo real, aunque parezca extraño, la apropiación a veces es no necesaria debido a que los procesos saben que no se pueden ejecutar durante periodos extensos, que por lo general realizan su trabajo y se bloquean con rapidez. La diferencia con los sistemas interactivos es que los sistemas de tiempo real sólo ejecutan programas destinados para ampliar la aplicación en cuestión. Los sistemas interactivos son de propósito general y pueden ejecutar programas arbitrarios que no sean cooperativos, o incluso malintencionados.

Metas de los algoritmos de planificación

Para poder diseñar un algoritmo de programación, es necesario tener cierta idea de lo que debe hacer un buen algoritmo. Algunos objetivos dependen del entorno (procesamiento por lotes, interactivo o de tiempo real), pero hay también algunos otros que son deseables en todos los casos. En la figura 2-39 se listan algunas metas. A continuación los analizaremos, uno a la vez.

Todos los sistemas

- Equidad - Otorgar a cada proceso una parte justa de la CPU
- Aplicación de políticas - Verificar que se lleven a cabo las políticas establecidas
- Balance - Mantener ocupadas todas las partes del sistema

Sistemas de procesamiento por lotes

- Rendimiento - Maximizar el número de trabajos por hora
- Tiempo de retorno - Minimizar el tiempo entre la entrega y la terminación
- Utilización de la CPU - Mantener ocupada la CPU todo el tiempo

Sistemas interactivos

- Tiempo de respuesta - Responder a las peticiones con rapidez
- Proporcionalidad - Cumplir las expectativas de los usuarios

Sistemas de tiempo real

- Cumplir con los plazos - Evitar perder datos
- Predictibilidad - Evitar la degradación de la calidad en los sistemas multimedia

Figura 2-39. Algunas metas del algoritmo de planificación bajo distintas circunstancias.

Bajo todas las circunstancias, la equidad es importante. Los procesos comparables deben obtener un servicio comparable. No es justo otorgar a un proceso mucho más tiempo de la CPU que a uno equivalente. Desde luego que las distintas categorías de procesos se pueden tratar de manera diferente. Considere el control de la seguridad y la administración de nómina en un centro computacional de un reactor nuclear.

La ejecución de las políticas del sistema está relacionada con la equidad. Si la política local es que los procesos de control de seguridad se ejecuten cada vez que lo deseen, aun si esto significa que la nómina se tardará 30 segundos más, el planificador tiene que asegurar que se ejecute esta política.

Otra meta general es mantener ocupadas todas las partes del sistema siempre que sea posible. Si la CPU y todos los dispositivos de E/S se pueden mantener en ejecución todo el tiempo, se realizará una mayor cantidad de trabajo por segundo que cuando algunos de los componentes están inactivos. Por ejemplo, en un sistema de procesamiento por lotes, el planificador controla cuáles trabajos se llevan a memoria para ejecutarlos. Es mejor tener algunos procesos limitados a CPU y algunos procesos limitados a E/S juntos en la memoria, que primero cargar y ejecutar todos los trabajos limitados a CPU y después, cuando terminen, cargar y ejecutar todos los trabajos limitados a E/S. Si se utiliza esta última estrategia, cuando los procesos ligados a CPU se ejecuten, lucharán por la CPU y el disco estará inactivo. Más adelante, cuando entren los trabajos ligados a E/S, lu-

charán por el disco y la CPU estará inactiva. Es mejor mantener todo el sistema en ejecución al mismo tiempo mediante una cuidadosa mezcla de procesos.

Los administradores de los centros grandes de cómputo que ejecutan muchos trabajos de procesamiento por lotes comúnmente se basan en tres métricas para verificar el desempeño de sus sistemas: rendimiento, tiempo de retorno y utilización de la CPU. El **rendimiento** es el número de trabajos por hora que completa el sistema. Considerando todos los detalles, es mejor terminar 50 trabajos por hora que terminar 40. El **tiempo de retorno** es el tiempo estadísticamente promedio desde el momento en que se envía un trabajo por lotes, hasta el momento en que se completa. Este parámetro mide cuánto tiempo tiene que esperar el usuario promedio la salida. He aquí la regla: lo pequeño es bello.

Un algoritmo de planificación que maximiza el rendimiento no necesariamente minimiza el tiempo de retorno. Por ejemplo, dada una mezcla de trabajos cortos y largos, un planificador que siempre ha ejecutado trabajos cortos y nunca ejecutó trabajos largos podría lograr un rendimiento excelente (muchos trabajos cortos por hora), pero a expensas de un terrible tiempo de retorno para los trabajos largos. Si los trabajos cortos llegaran a un tiempo bastante estable, los trabajos largos tal vez nunca se ejecutarían, con lo cual el tiempo de respuesta promedio se volvería infinito, a la vez que se lograría un rendimiento alto.

La utilización de la CPU se emplea a menudo como métrica en los sistemas de procesamiento por lotes. En realidad, no es una buena métrica. Lo que importa es cuántos trabajos por hora salen del sistema (rendimiento) y cuánto tiempo se requiere para obtener un trabajo de vuelta (tiempo de retorno). Con la utilización de la CPU como métrica es como clasificar a los autos con base en cuántas veces por hora gira el motor. Por otro lado, es útil poder detectar cuando el uso de la CPU está llegando a 100%, para saber cuándo es momento de obtener más poder de cómputo.

Para los sistemas interactivos se aplican distintas metas. La más importante es minimizar el **tiempo de respuesta**; es decir, el tiempo que transcurre entre emitir un comando y obtener el resultado. En una computadora personal en la que se ejecuta un proceso en segundo plano (por ejemplo, leer y almacenar correo electrónico de la red), una petición del usuario para iniciar un programa o abrir un archivo debe tener precedencia sobre el trabajo en segundo plano. Hacer que todas las peticiones interactivas se atiendan primero se percibirá como un buen servicio.

Una cuestión algo relacionada es lo que podría denominarse **proporcionalidad**. Los usuarios tienen una idea inherente (pero a menudo incorrecta) acerca de cuánto deben tardar las cosas. Cuando una petición que se percibe como compleja tarda mucho tiempo, los usuarios aceptan eso, pero cuando una petición que se percibe como simple tarda mucho tiempo, los usuarios se irritan. Por ejemplo, si al hacer clic en un icono que empieza a enviar un fax el sistema tarda 60 segundos en completar la oración, el usuario probablemente aceptará eso como un gaje del oficio, debido a que no espera que un fax se envíe en 5 segundos.

Por otro lado, cuando un usuario hace clic en el icono que interrumpe la conexión telefónica una vez que se ha enviado el fax, tiene diferentes expectativas. Si no se ha completado después de 30 segundos, es probable que el usuario esté sumamente enojado y después de 60 segundos estará echando espuma por la boca. Este comportamiento se debe a la percepción común de los usuarios de que se *supone* que hacer una llamada telefónica y enviar un fax requiere mucho más tiempo que sólo colgar el teléfono. En algunos casos (como éste) el planificador no puede hacer nada acerca del

tiempo de respuesta, pero en otros casos sí puede, en especial cuando el retraso se debe a una mala elección en el orden de los procesos.

Los sistemas de tiempo real tienen propiedades distintas de los sistemas interactivos y por ende diferentes metas de planificación. Se caracterizan por tener tiempos límite que deben (o deberían) cumplirse. Por ejemplo, si una computadora está controlando un dispositivo que produce datos a un paso regular, al no ejecutar el proceso de recolección de datos a tiempo se podría perder información. Por ende, la principal necesidad en un sistema de tiempo real es cumplir todos los tiempos límite (o la mayoría).

En algunos sistemas de tiempo real, en especial los que involucran el uso de multimedia, la predictibilidad es importante. No es fatal fallar en un tiempo límite, pero si el procesador de audio se ejecuta con muchos errores, la calidad del sonido se deteriorará con rapidez. El video también cuenta, pero el oído es mucho más sensible a la perturbación que el ojo. Para evitar este problema, la planificación de procesos debe ser altamente predecible y regular. En este capítulo estudiaremos los algoritmos de planificación de procesamiento por lotes e interactivos, pero aplazaremos la mayor parte de nuestro estudio de la planificación de tiempo real para cuando analicemos los sistemas operativos multimedia en el capítulo 7.

2.4.2 Planificación en sistemas de procesamiento por lotes

Ahora es tiempo de pasar de las cuestiones de planificación en general a los algoritmos de planificación específicos. En esta sección analizaremos los algoritmos que se utilizan en sistemas de procesamiento por lotes. En las siguientes secciones examinaremos los sistemas interactivos y de tiempo real. Vale la pena aclarar que algunos algoritmos se utilizan tanto en los sistemas de procesamiento por lotes como interactivos. Más adelante estudiaremos estos algoritmos.

Primero en entrar, primero en ser atendido

Probablemente el más simple de todos los algoritmos de planificación es el de tipo **primero en entrar, primero en ser atendido** (FCFS, *First-Come, First-Served*) no apropiativo. Con este algoritmo, la CPU se asigna a los procesos en el orden en el que la solicitan. En esencia hay una sola cola de procesos listos. Cuando el primer trabajo entra al sistema desde el exterior en la mañana, se inicia de inmediato y se le permite ejecutarse todo el tiempo que desee. No se interrumpe debido a que se ha ejecutado demasiado tiempo. A medida que van entrando otros trabajos, se colocan al final de la cola. Si el proceso en ejecución se bloquea, el primer proceso en la cola se ejecuta a continuación. Cuando un proceso bloqueado pasa al estado listo, al igual que un trabajo recién llegado, se coloca al final de la cola.

La gran fuerza de este algoritmo es que es fácil de comprender e igualmente sencillo de programar. También es equitativo, en el mismo sentido en que es equitativo asignar los escasos boletos para eventos deportivos o conciertos a las personas que están dispuestas a permanecer en la línea desde las 2 A.M. Con este algoritmo, una sola lista ligada lleva la cuenta de todos los procesos listos. Para elegir un proceso a ejecutar sólo se requiere eliminar uno de la parte frontal de la cola. Para agregar un nuevo trabajo o desbloquear un proceso sólo hay que adjuntarlo a la parte final de la cola. ¿Qué podría ser más simple de comprender y de implementar?

Por desgracia, el algoritmo tipo “primero en entrar, primero en ser atendido” también tiene una importante desventaja. Suponga que hay un proceso ligado a los cálculos que se ejecuta durante 1 segundo en cierto momento, junto con muchos procesos limitados a E/S que utilizan poco tiempo de la CPU, pero cada uno de ellos tiene que realizar 1000 lecturas de disco para completarse. El proceso limitado a los cálculos se ejecuta durante 1 segundo y después lee un bloque de disco. Ahora se ejecutan todos los procesos de E/S e inician lecturas de disco. Cuando el proceso limitado a los cálculos obtiene su bloque de disco, se ejecuta por otro segundo, seguido de todos los procesos limitados a E/S en una rápida sucesión.

El resultado neto es que cada proceso limitado a E/S llega a leer 1 bloque por segundo y requerirá 1000 segundos para completarse. Con un algoritmo de planificación que se apropió del proceso limitado a los cálculos cada 10 mseg, los procesos limitados a E/S terminarían en 10 segundos en vez de 1000 segundos y sin quitar mucha velocidad al proceso limitado a los cálculos.

El trabajo más corto primero

Ahora analicemos otro algoritmo de procesamiento por lotes no apropiativo, que supone que los tiempos de ejecución se conocen de antemano. Por ejemplo, en una compañía de seguros las personas pueden predecir con bastante precisión cuánto tiempo se requerirá para ejecutar un lote de 1000 reclamaciones, ya que se realiza un trabajo similar cada día. Cuando hay varios trabajos de igual importancia esperando a ser iniciados en la cola de entrada, el planificador selecciona **el trabajo más corto primero** (SJF, *Shortest Job First*). Analice la figura 2-40. Aquí encontramos cuatro trabajos (*A*, *B*, *C* y *D*) con tiempos de ejecución de 8, 4, 4 y 4 minutos, respectivamente. Al ejecutarlos en ese orden, el tiempo de respuesta para *A* es de 8 minutos, para *B* es de 12 minutos, para *C* es de 16 minutos y para *D* es de 20 minutos, para un promedio de 14 minutos.



Figura 2-40. Un ejemplo de planificación tipo el trabajo más corto primero. (a) Ejecución de cuatro trabajos en el orden original. (b) Ejecución de cuatro trabajos en el orden del tipo “el trabajo más corto primero”.

Ahora consideremos ejecutar estos cuatro trabajos usando el trabajo más corto primero, como se muestra en la figura 2-40(b). Los tiempos de respuesta son ahora de 4, 8, 12 y 20 minutos, para un promedio de 11 minutos. Probablemente sea óptimo el algoritmo tipo el trabajo más corto primero. Considere el caso de cuatro trabajos, con tiempos de ejecución de a , b , c y d , respectivamente. El primer trabajo termina en el tiempo a , el segundo termina en el tiempo $a + b$, y así en lo sucesivo. El tiempo promedio de respuesta es $(4a + 3b + 2c + d)/4$. Está claro que a contribuye más al promedio que los otros tiempos, por lo que debe ser el trabajo más corto, con b a continuación, después c y por último d como el más largo, ya que sólo afecta a su tiempo de retorno. El mismo argumento se aplica con igual facilidad a cualquier número de trabajos.

Vale la pena observar que el trabajo más corto primero es sólo óptimo cuando todos los trabajos están disponibles al mismo tiempo. Como ejemplo contrario, considere cinco trabajos (del *A* al *E*) con tiempos de ejecución de 2, 4, 1, 1 y 1, respectivamente. Sus tiempos de llegada son 0, 0, 3, 3 y 3. Al principio sólo se pueden elegir *A* o *B*, ya que los otros tres trabajos no han llegado todavía. Utilizando el trabajo más corto primero, ejecutaremos los trabajos en el orden *A, B, C, D, E* para un tiempo de espera promedio de 4.6. Sin embargo, al ejecutarlos en el orden *B, C, D, E, A* hay una espera promedio de 4.4.

El menor tiempo restante a continuación

Una versión apropiativa del algoritmo tipo el trabajo más corto primero es **el menor tiempo restante a continuación** (SRTN, *Shortest Remaining Time Next*). Con este algoritmo, el planificador siempre selecciona el proceso cuyo tiempo restante de ejecución sea el más corto. De nuevo, se debe conocer el tiempo de ejecución de antemano. Cuando llega un nuevo trabajo, su tiempo total se compara con el tiempo restante del proceso actual. Si el nuevo trabajo necesita menos tiempo para terminar que el proceso actual, éste se suspende y el nuevo trabajo se inicia. Ese esquema permite que los trabajos cortos nuevos obtengan un buen servicio.

2.4.3 Planificación en sistemas interactivos

Ahora analizaremos algunos algoritmos que se pueden utilizar en sistemas interactivos. Estos algoritmos son comunes en computadoras personales, servidores y en otros tipos de sistemas también.

Planificación por turno circular

Uno de los algoritmos más antiguos, simples, equitativos y de mayor uso es el de **turno circular** (*round-robin*). A cada proceso se le asigna un intervalo de tiempo, conocido como **quántum**, durante el cual se le permite ejecutarse. Si el proceso se sigue ejecutando al final del cuanto, la CPU es apropiada para dársela a otro proceso. Si el proceso se bloquea o termina antes de que haya transcurrido el cuántum, la conmutación de la CPU se realiza cuando el proceso se bloquea, desde luego. Es fácil implementar el algoritmo de turno circular. Todo lo que el realizador necesita es mantener una lista de procesos ejecutables, como se muestra en la figura 2-41(a). Cuando el proceso utiliza su cuántum, se coloca al final de la lista, como se muestra en la figura 2-41(b).

La única cuestión interesante con el algoritmo de turno circular es la longitud del cuántum. Para conmutar de un proceso a otro se requiere cierta cantidad de tiempo para realizar la administración: guardar y cargar tanto registros como mapas de memoria, actualizar varias tablas y listas, vaciar y recargar la memoria caché y así sucesivamente. Suponga que esta **conmutación de proceso** o **conmutación de contexto** (como algunas veces se le llama) requiere 1 mseg, incluyendo el cambio de los mapas de memoria, el vaciado y recarga de la caché, etc. Suponga además que el cuántum se establece a 4 mseg. Con estos parámetros, después de realizar 4 mseg de trabajo útil, la CPU tendrá que gastar (es decir, desperdiciar) 1 mseg en la conmutación de procesos. Por ende,



Figura 2-41. Planificación de turno circular. (a) La lista de procesos ejecutables. (b) La lista de procesos ejecutables una vez que *B* utiliza su cuántum.

20 por ciento de la CPU se desperdiciará por sobrecarga administrativa. Sin duda, esto es demasiado.

Para mejorar la eficiencia de la CPU, podríamos establecer el cuántum a (por decir) 100 mseg. Ahora el tiempo desperdiciado es sólo de 1 por ciento. Pero considere lo que ocurre en un sistema servidor si llegan 50 peticiones dentro de un intervalo de tiempo muy corto y con requerimientos muy variables de la CPU. Se colocarán cincuenta procesos en la lista de procesos ejecutables. Si la CPU está inactiva, el primero empezará de inmediato, el segundo tal vez no inicie sino hasta 100 mseg después y así en lo sucesivo. El último desafortunado tal vez tenga que esperar 5 segundos para poder tener una oportunidad, suponiendo que los demás utilizan sus cuántums completos. La mayoría de los usuarios percibirán como lenta una respuesta de 5 segundos a un comando corto. Esta situación es muy mala si algunas de las peticiones cerca del final de la cola requirieron sólo unos cuantos milisegundos de tiempo de la CPU. Con un cuántum corto, hubieran obtenido un mejor servicio.

Otro factor es que si al cuántum se le asigna un tiempo más largo que la ráfaga promedio de la CPU, la apropiación no ocurrirá con mucha frecuencia. En vez de ello, la mayoría de los procesos realizarán una operación de bloqueo antes de que el cuántum se agote, ocasionando una conmutación de proceso. Al eliminar la apropiación mejora el rendimiento, debido a que las conmutaciones de procesos sólo ocurrirán cuando sea lógicamente necesario; es decir, cuando un proceso se bloquee y no pueda continuar.

La conclusión se puede formular de la siguiente manera: si se establece el cuántum demasiado corto se producen demasiadas conmutaciones de procesos y se reduce la eficiencia de la CPU, pero si se establece demasiado largo se puede producir una mala respuesta a las peticiones interactivas cortas. A menudo, un cuántum con un valor entre 20 y 50 mseg constituye una solución razonable.

Planificación por prioridad

La planificación por turno circular hace la suposición implícita de que todos los procesos tienen igual importancia. Con frecuencia, las personas que poseen y operan computadoras multiusuario tienen diferentes ideas en cuanto a ese aspecto. Por ejemplo, en una universidad el orden jerárquico puede ser: primero los decanos, después los profesores, secretarias, conserjes y por último los estudiantes. La necesidad de tomar en cuenta los factores externos nos lleva a la **planificación por prioridad**. La idea básica es simple: a cada proceso se le asigna una prioridad y el proceso ejecutable con la prioridad más alta es el que se puede ejecutar.

Incluso hasta en una PC con un solo propietario puede haber varios procesos, algunos de ellos más importantes que los demás. Por ejemplo, un proceso demonio que envía correo electrónico en segundo plano debería recibir una menor prioridad que un proceso que muestra una película de video en la pantalla en tiempo real.

Para evitar que los procesos con alta prioridad se ejecuten de manera indefinida, el planificador puede reducir la prioridad del proceso actual en ejecución en cada pulso del reloj (es decir, en cada interrupción del reloj). Si esta acción hace que su prioridad se reduzca a un valor menor que la del proceso con la siguiente prioridad más alta, ocurre una conmutación de procesos. De manera alternativa, a cada proceso se le puede asignar un cuántum de tiempo máximo que tiene permitido ejecutarse. Cuando este cuántum se utiliza, el siguiente proceso con la prioridad más alta recibe la oportunidad de ejecutarse.

A las prioridades se les pueden asignar procesos en forma estática o dinámica. En una computadora militar, los procesos iniciados por los generales podrían empezar con una prioridad de 100, aquéllos iniciados por los coroneles con 90, los mayores con 80, los capitanes con 70, los tenientes con 60 y así sucesivamente. De manera alternativa, en un centro computacional comercial los trabajos con prioridad alta podrían costar \$100 por hora, los de prioridad media \$75 y los de prioridad baja \$50 por hora. El sistema UNIX tiene un comando llamado *nice*, el cual permite a un usuario reducir de manera voluntaria la prioridad de su proceso, para que sea agradable a los otros usuarios. Nadie lo utiliza.

El sistema también puede asignar las prioridades en forma dinámica para lograr ciertos objetivos. Por ejemplo, algunos procesos están muy limitados a E/S y gastan la mayor parte de su tiempo esperando a que la E/S se complete. Cada vez que un proceso así desea la CPU, debe recibirla de inmediato para dejar que inicie su siguiente petición de E/S, que a su vez puede proceder en paralelo con otro proceso que se encuentre realizando cálculos. Hacer que el proceso limitado a E/S espere mucho tiempo por la CPU sólo significa que estará ocupando memoria por un tiempo innecesariamente largo. Un algoritmo simple para dar buen servicio a los procesos limitados a E/S es establecer la prioridad a $1/f$, en donde f es la fracción del último cuántum que utilizó un proceso. Un proceso que sólo utilizó 1 mseg de su cuántum de 50 mseg obtendría una prioridad de 50, mientras que un proceso que se ejecutara durante 25 mseg antes de bloquearse recibiría la prioridad 2 y un proceso que utilizara el cuántum completo recibiría la prioridad 1.

A menudo es conveniente agrupar los procesos en clases de prioridad y utilizar la planificación por prioridad entre las clases, pero la planificación por turno circular dentro de cada clase. La figura 2-42 muestra un sistema con cuatro clases de prioridad. El algoritmo de programación es el siguiente: mientras que haya procesos ejecutables en la clase de prioridad 4, sólo ejecutar cada uno por un cuanto, por turno circular y nunca preocuparse por las clases con menor prioridad. Si la clase de prioridad 4 está vacía, entonces ejecutar los procesos de la clase 3 por turno circular. Si las clases 4 y 3 están vacías, entonces ejecutar la clase 2 por turno circular y así sucesivamente. Si las prioridades no se ajustan de manera ocasional, todas las clases de menor prioridad pueden *morir de hambre* (sin tiempo de CPU).

Múltiples colas

Uno de los primeros planificadores por prioridad estaba en CTSS, el Sistema de tiempo compartido compatible del M.I.T. que se ejecutaba en la IBM 7094 (Corbató y colaboradores, 1962). CTSS

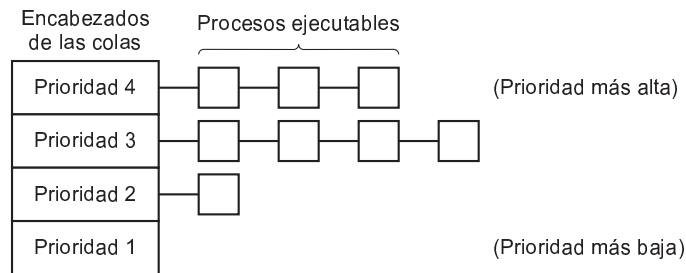


Figura 2-42. Un algoritmo de planificación con cuatro clases de prioridad.

tenía el problema de que la conmutación de procesos era muy lenta, debido a que la 7094 sólo podía contener un proceso en la memoria. Cada conmutación de procesos ocasionaba intercambiar (*swapping*) el proceso actual al disco y leer uno nuevo del disco. Los diseñadores de CTSS se dieron cuenta rápidamente de que era más eficiente dar a los procesos limitados a CPU un cuántum largo de vez en cuando, en vez de darles cuántums pequeños con frecuencia (para reducir el intercambio). Por otro lado, al proporcionar a todos los procesos un cuántum largo se obtendría un tiempo de respuesta pobre, como ya hemos visto. Su solución fue la de establecer clases de prioridades. Los procesos en la clase más alta se ejecutaban durante un cuántum. Los procesos en la siguiente clase más alta se ejecutaban por dos cuántums. Los procesos en la siguiente clase se ejecutaban por cuatro cuántums, y así sucesivamente. Cada vez que un proceso utilizaba todos los cuántums que tenía asignados, se movía una clase hacia abajo en la jerarquía.

Como ejemplo, considere un proceso que necesita realizar cálculos en forma continua durante 100 cuántums. Al inicio recibe un cuántum y después se intercambia con otro proceso en disco. La siguiente vez recibirá dos cuántums antes de que se vuelva a intercambiar. En las siguientes ejecuciones obtendrá 4, 8, 12, 16, 32 y 64 cuántums, aunque habrá utilizado sólo 37 de los 64 cuántums finales para completar su trabajo. Sólo 7 intercambios (incluyendo la carga inicial) se requerirán, en vez de 100, si se utiliza sólo un algoritmo por turno circular. Lo que es más, a medida que el proceso se hunde más y más en las colas de prioridad, se ejecutará con cada vez menos frecuencia, guardando la CPU para procesos cortos e interactivos.

La siguiente política se adoptó para evitar que un proceso, que requería ejecutarse durante un tiempo largo cuando empezó por primera vez pero se volvió interactivo más adelante, fuera castigado para siempre. Cada vez que se escribía un retorno de carro (tecla Intro o Enter) en una terminal, el proceso que pertenecía a esa terminal se movía a la clase de mayor prioridad, con base en la suposición de que estaba a punto de volverse interactivo. Un buen día, algún usuario con un proceso muy limitado a la CPU descubrió que con sólo sentarse en la terminal y teclear retornos de carro al azar durante unos cuantos segundos, su tiempo de respuesta mejoraba en forma considerable. Se lo contó a todos sus amigos. Moraleja de la historia: llegar a lo correcto en la práctica es más difícil que en la teoría.

Se han utilizado muchos otros algoritmos para asignar procesos a las clases de prioridades. Por ejemplo, el influyente sistema XDS 940 (Lampson, 1968) construido en Berkeley tenía cuatro clases de prioridad: terminal, E/S, cuántum corto y cuántum largo. Cuando un proceso que había estado esperando la entrada de terminal por fin se despertaba, pasaba a la clase de mayor prioridad

(terminal). Cuando un proceso en espera de un bloque de disco pasaba al estado listo, se enviaba a la segunda clase. Cuando a un proceso que estaba todavía en ejecución se le agotaba su cuántum, al principio se colocaba en la tercera clase. No obstante, si un proceso utilizaba todo su cuántum demasiadas veces seguidas sin bloquearse en espera de la terminal o de otro tipo de E/S, se movía hacia abajo hasta la última cola. Muchos otros sistemas utilizan algo similar para favorecer a los usuarios y procesos interactivos en vez de los que se ejecutan en segundo plano.

El proceso más corto a continuación

Como el algoritmo tipo el trabajo más corto primero siempre produce el tiempo de respuesta promedio mínimo para los sistemas de procesamiento por lotes, sería bueno si se pudiera utilizar para los procesos interactivos también. Hasta cierto grado, esto es posible. Por lo general, los procesos interactivos siguen el patrón de esperar un comando, ejecutarlo, esperar un comando, ejecutarlo, etcétera. Si consideramos la ejecución de cada comando como un “trabajo” separado, entonces podríamos minimizar el tiempo de respuesta total mediante la ejecución del más corto primero. El único problema es averiguar cuál de los procesos actuales ejecutables es el más corto.

Un método es realizar estimaciones con base en el comportamiento anterior y ejecutar el proceso con el tiempo de ejecución estimado más corto. Suponga que el tiempo estimado por cada comando para cierta terminal es T_0 . Ahora suponga que su siguiente ejecución se mide como T_1 . Podríamos actualizar nuestra estimación mediante una suma ponderada de estos dos números, es decir, $aT_0 + (1 - a)T_1$. Por medio de la elección de a podemos decidir hacer que el proceso de estimación olvide las ejecuciones anteriores rápidamente o que las recuerde por mucho tiempo. Con $a = 1/2$, obtenemos estimaciones sucesivas de

$$T_0 \quad T_0/2 + T_1/2 \quad T_0/4 + T_1/4 + T_2/2 \quad T_0/8 + T_1/8 + T_2/4 + T_3/2$$

Después de tres nuevas ejecuciones, el peso de T_0 en la nueva estimación se ha reducido a $1/8$.

La técnica de estimar el siguiente valor en una serie mediante la obtención del promedio ponderado del valor actual medido y la estimación anterior se conoce algunas veces como **envejecimiento**. Se aplica en muchas situaciones en donde se debe realizar una predicción con base en valores anteriores. En especial, el envejecimiento es fácil de implementar cuando $a = 1/2$. Todo lo que se necesita es sumar el nuevo valor a la estimación actual y dividir la suma entre 2 (mediante el desplazamiento de un bit a la derecha).

Planificación garantizada

Un método completamente distinto para la planificación es hacer promesas reales a los usuarios acerca del rendimiento y después cumplirlas. Una de ellas, que es realista y fácil de cumplir es: si hay n usuarios conectados mientras usted está trabajando, recibirá aproximadamente $1/n$ del poder de la CPU. De manera similar, en un sistema de un solo usuario con n procesos en ejecución, mientras no haya diferencias, cada usuario debe obtener $1/n$ de los ciclos de la CPU. Eso parece bastante justo.

Para cumplir esta promesa, el sistema debe llevar la cuenta de cuánta potencia de CPU ha tenido cada proceso desde su creación. Después calcula cuánto poder de la CPU debe asignarse a cada proceso, a saber el tiempo desde que se creó dividido entre n . Como la cantidad de tiempo de CPU que ha tenido cada proceso también se conoce, es simple calcular la proporción de tiempo de CPU que se consumió con el tiempo de CPU al que cada proceso tiene derecho. Una proporción de 0.5 indica que un proceso solo ha tenido la mitad del tiempo que debería tener, y una proporción de 2.0 indica que un proceso ha tenido el doble de tiempo del que debería tener. Entonces, el algoritmo es para ejecutar el proceso con la menor proporción hasta que se haya desplazado por debajo de su competidor más cercano.

Planificación por sorteo

Aunque hacer promesas a los usuarios y cumplirlas es una buena idea, es algo difícil de implementar. Sin embargo, se puede utilizar otro algoritmo para producir resultados similares con una implementación mucho más sencilla. Este algoritmo se conoce como **planificación por sorteo** (Waldspurger y Weihl, 1994).

La idea básica es dar a los procesos boletos de lotería para diversos recursos del sistema, como el tiempo de la CPU. Cada vez que hay que tomar una decisión de planificación, se selecciona un boleto de lotería al azar y el proceso que tiene ese boleto obtiene el recurso. Cuando se aplica a la planificación de la CPU, el sistema podría realizar un sorteo 50 veces por segundo y cada ganador obtendría 20 mseg de tiempo de la CPU como premio.

Parafraseando a George Orwell: “Todos los procesos son iguales, pero algunos son más iguales que otros”. Los procesos más importantes pueden recibir boletos adicionales, para incrementar su probabilidad de ganar. Si hay 100 boletos repartidos y un proceso tiene 20 de ellos, tendrá una probabilidad de 20 por ciento de ganar cada sorteo. A la larga, recibirá un 20 por ciento del tiempo de la CPU. En contraste a un planificador por prioridad, en donde es muy difícil establecer lo que significa tener una prioridad de 40, aquí la regla es clara: un proceso que contenga una fracción f de los boletos recibirá aproximadamente una fracción f del recurso en cuestión.

La planificación por lotería tiene varias propiedades interesantes. Por ejemplo, si aparece un nuevo proceso y recibe algunos boletos, en el siguiente sorteo tendrá la oportunidad de ganar en proporción al número de boletos que tenga. En otras palabras, la planificación por lotería tiene un alto grado de respuesta.

Los procesos cooperativos pueden intercambiar boletos si lo desean. Por ejemplo, cuando un proceso cliente envía un mensaje a un proceso servidor y después se bloquea, puede dar todos sus boletos al servidor para incrementar la probabilidad de que éste se ejecute a continuación. Cuando el servidor termina, devuelve los boletos para que el cliente se pueda ejecutar de nuevo. De hecho, en ausencia de clientes, los servidores no necesitan boletos.

La planificación por sorteo se puede utilizar para resolver problemas que son difíciles de manejar con otros métodos. Un ejemplo es un servidor de video en el que varios procesos están enviando flujos continuos de video a sus clientes, pero enviando sus cuadros a distintas velocidades. Suponga que el proceso necesita cuadros a 10, 20 y 25 cuadros por segundo. Al asignar a estos procesos 10, 20 y 25 boletos, respectivamente, dividirán de manera automática el tiempo de la CPU en la proporción correcta aproximada; es decir: 10 : 20 : 25.

Planificación por partes equitativas

Hasta ahora hemos asumido que cada proceso se planifica por su cuenta, sin importar quién sea su propietario. Como resultado, si el usuario 1 inicia 9 procesos y el usuario 2 inicia 1 proceso, con la planificación por turno circular o por prioridades iguales, el usuario 1 obtendrá 90 por ciento del tiempo de la CPU y el usuario 2 sólo recibirá 10 por ciento.

Para evitar esta situación, algunos sistemas toman en consideración quién es el propietario de un proceso antes de planificarlo. En este modelo, a cada usuario se le asigna cierta fracción de la CPU y el planificador selecciona procesos de tal forma que se cumpla con este modelo. Por ende, si a dos usuarios se les prometió 50 por ciento del tiempo de la CPU para cada uno, eso es lo que obtendrán sin importar cuántos procesos tengan en existencia.

Como ejemplo, considere un sistema con dos usuarios, y a cada uno de los cuales se les prometió 50 por ciento de la CPU. El usuario 1 tiene cuatro procesos (*A*, *B*, *C* y *D*) y el usuario 2 sólo tiene 1 proceso (*E*). Si se utiliza la planificación por turno circular, una posible secuencia de planificación que cumple con todas las restricciones es:

A E B E C E D E A E B E C E D E ...

Por otro lado, si el usuario 1 tiene derecho al doble de tiempo de la CPU que el usuario 2, podríamos obtener la siguiente secuencia:

A B E C D E A B E C D E ...

Desde luego que existen muchas otras posibilidades, y se pueden explotar dependiendo de cuál sea la noción de equidad.

2.4.4 Planificación en sistemas de tiempo real

En un sistema de **tiempo real**, el tiempo desempeña un papel esencial. Por lo general, uno o más dispositivos físicos externos a la computadora generan estímulo y la computadora debe reaccionar de manera apropiada a ellos dentro de cierta cantidad fija de tiempo. Por ejemplo, la computadora en un reproductor de disco compacto recibe los bits a medida que provienen de la unidad y debe convertirlos en música, en un intervalo de tiempo muy estrecho. Si el cálculo tarda demasiado, la música tendrá un sonido peculiar. Otros sistemas de tiempo real son el monitoreo de pacientes en una unidad de cuidados intensivos de un hospital, el autopiloto en una aeronave y el control de robots en una fábrica automatizada. En todos estos casos, tener la respuesta correcta pero demasiado tarde es a menudo tan malo como no tenerla.

En general, los sistemas de tiempo real se categorizan como de **tiempo real duro**, lo cual significa que hay tiempos límite absolutos que se deben cumplir, y como de **tiempo real suave**, lo cual significa que no es conveniente fallar en un tiempo límite en ocasiones, pero sin embargo es tolerable. En ambos casos, el comportamiento en tiempo real se logra dividiendo el programa en varios procesos, donde el comportamiento de cada uno de éstos es predecible y se conoce de antemano. Por lo general, estos procesos tienen tiempos de vida cortos y pueden ejecutarse hasta completarse en mucho menos de 1 segundo. Cuando se detecta un evento externo, es responsabilidad del planificador planificar los procesos de tal forma que se cumpla con todos los tiempos límite.

Los eventos a los que puede llegar a responder un sistema de tiempo real se pueden categorizar como **periódicos** (que ocurren a intervalos regulares) o **aperiódicos** (que ocurren de manera impredecible). Tal vez un sistema tenga que responder a varios flujos de eventos periódicos. Dependiendo de cuánto tiempo requiera cada evento para su procesamiento, tal vez ni siquiera sea posible manejarlos a todos. Por ejemplo, si hay m eventos periódicos y el evento i ocurre con el periodo P_i y requiere C_i segundos de tiempo de la CPU para manejar cada evento, entonces la carga sólo se podrá manejar si

$$\sum_{i=1}^m \frac{C_i}{P_i} \leq 1$$

Se dice que un sistema de tiempo real que cumple con este criterio es **planificable**.

Como ejemplo, considere un sistema de tiempo real con tres eventos periódicos, con periodos de 100, 200 y 500 mseg, respectivamente. Si estos eventos requieren 50, 30 y 100 mseg de tiempo de la CPU por evento, respectivamente, el sistema es planificable debido a que $0.5 + 0.15 + 0.2 < 1$. Si se agrega un cuarto evento con un periodo de 1 segundo, el sistema seguirá siendo planificable mientras que este evento no requiera más de 150 mseg de tiempo de la CPU por evento. En este cálculo está implícita la suposición de que la sobrecarga por la conmutación de contexto es tan pequeña que se puede ignorar.

Los algoritmos de planificación en tiempo real pueden ser estáticos o dinámicos. Los primeros toman sus decisiones de planificación antes de que el sistema empiece a ejecutarse. Los segundos lo hacen durante el tiempo de ejecución. La planificación estática sólo funciona cuando hay información perfecta disponible de antemano acerca del trabajo que se va a realizar y los tiempos límite que se tienen que cumplir. Los algoritmos de planificación dinámicos no tienen estas restricciones. Aplazaremos nuestro estudio de algoritmos específicos hasta el capítulo 7, donde trataremos los sistemas de multimedia en tiempo real.

2.4.5 Política contra mecanismo

Hasta ahora hemos supuesto tácitamente que todos los procesos en el sistema pertenecen a distintos usuarios y por lo tanto compiten por la CPU. Aunque esto es a menudo cierto, algunas veces sucede que un proceso tiene muchos hijos ejecutándose bajo su control. Por ejemplo, un proceso de un sistema de administración de bases de datos puede tener muchos hijos. Cada hijo podría estar trabajando en una petición distinta o cada uno podría tener cierta función específica que realizar (análisis de consultas o acceso al disco, por ejemplo). Es por completo posible que el proceso principal tenga una idea excelente acerca de cuál de sus hijos es el más importante (o que requiere tiempo con más urgencia) y cuál es el menos importante. Por desgracia, ninguno de los planificadores antes descritos acepta entrada de los procesos de usuario acerca de las decisiones de planificación. Como resultado, raras veces el planificador toma la mejor decisión.

La solución a este problema es separar el **mecanismo de planificación** de la **política de planificación**, un principio establecido desde hace tiempo (Levin y colaboradores, 1975). Esto significa que el algoritmo de planificación está parametrizado de cierta forma, pero los procesos de usuario pueden llenar los parámetros. Consideremos el ejemplo de la base de datos una vez más.

Suponga que el kernel utiliza un algoritmo de planificación por prioridad, pero proporciona una llamada al sistema mediante la cual un proceso puede establecer (y modificar) las prioridades de sus hijos. De esta forma, el padre puede controlar con detalle la forma en que se planifican sus hijos, aun y cuando éste no se encarga de la planificación. Aquí el mecanismo está en el kernel, pero la política se establece mediante un proceso de usuario.

2.4.6 Planificación de hilos

Cuando varios procesos tienen múltiples hilos cada uno, tenemos dos niveles de paralelismo presentes: procesos e hilos. La planificación en tales sistemas difiere en forma considerable, dependiendo de si hay soporte para hilos a nivel usuario o para hilos a nivel kernel (o ambos).

Consideremos primero los hilos a nivel usuario. Como el kernel no está consciente de la existencia de los hilos, opera en la misma forma de siempre: selecciona un proceso, por decir *A*, y otorga a este proceso el control de su cuántum. El planificador de hilos dentro de *A* decide cuál hilo ejecutar, por decir *A1*. Como no hay interrupciones de reloj para multiprogramar hilos, este hilo puede continuar ejecutándose todo el tiempo que quiera. Si utiliza todo el cuántum del proceso, el kernel seleccionará otro proceso para ejecutarlo.

Cuando el proceso *A* por fin se ejecute de nuevo, el hilo *A1* continuará su ejecución. Seguirá consumiendo todo el tiempo de *A* hasta que termine. Sin embargo, su comportamiento antisocial no afectará a los demás procesos. Éstos recibirán lo que el planificador de procesos considere que es su parte apropiada, sin importar lo que esté ocurriendo dentro del proceso *A*.

Ahora considere el caso en el que los hilos de *A* tienen relativamente poco trabajo que realizar por cada ráfaga de la CPU; por ejemplo, 5 mseg de trabajo dentro de un cuántum de 50 mseg. En consecuencia, cada uno se ejecuta por unos instantes y después entrega la CPU al planificador de hilos. Esto podría producir la secuencia *A1, A2, A3, A1, A2, A3, A1, A2, A3, A1* antes de que el kernel conmute al proceso *B*. Esta situación se ilustra en la figura 2-43(a).

El algoritmo de planificación utilizado por el sistema en tiempo de ejecución puede ser cualquiera de los antes descritos. En la práctica, los algoritmos de planificación por turno circular y de planificación por prioridad son los más comunes. La única restricción es la ausencia de un reloj para interrumpir a un proceso que se ha ejecutado por mucho tiempo.

Ahora considere la situación con hilos a nivel kernel. Aquí el kernel selecciona un hilo específico para ejecutarlo. No tiene que tomar en cuenta a cuál proceso pertenece el hilo, pero puede hacerlo si lo desea. El hilo recibe un cuántum y se suspende obligatoriamente si se excede de este cuántum. Con un cuántum de 50 mseg pero hilos que se bloquean después de 5 mseg, el orden de los hilos para cierto periodo de 30 mseg podría ser *A1, B1, A2, B2, A3, B3* algo que no sería posible con estos parámetros e hilos a nivel usuario. Esta situación se ilustra en forma parcial en la figura 2-43(b).

Una diferencia importante entre los hilos a nivel usuario y los hilos a nivel kernel es el rendimiento. Para realizar una conmutación de hilos con hilos a nivel usuario se requiere de muchas instrucciones de máquina. Con hilos a nivel kernel se requiere una conmutación de contexto total, cambiar el mapa de memoria e invalidar la caché, lo cual es varias órdenes de magnitud más lento. Por otro lado, con los hilos a nivel kernel, cuando un hilo se bloquea en espera de E/S no se suspende todo el proceso, como con los hilos a nivel usuario.